

Crop Disease Prediction System

A Mini Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

Labdhi Vohra
EN23CS301553

Lakshita Patel
EN23CS301557

Lakshita Porwal
EN23CS301558

Under the Guidance of

Prof. Mahesh Patidar
Prof. Madan Pal Singh



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE – 453331

APRIL-2026

CROP DISEASE PREDICTION SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement
of the

**BACHELOR OF TECHNOLOGY in
COMPUTER SCIENCE & ENGINEERING**

BY

Labdhi Vohra (EN23CS301553)

Lakshita Patel (EN23CS301557)

Lakshita Porwal (EN23CS301558)

Under the Guidance of

Dr. Madan Pal Singh & Dr. Mahesh Patidar



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE - 453331

Jan-Jun 2026

Report Approval

The project work "**Crop Disease Prediction System**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: _____

Designation: _____

Affiliation: _____

External Examiner

Name: _____

Designation: _____

Affiliation: _____

Declaration

We hereby declare that the project entitled "**Crop Disease Prediction System**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of Dr. Madan Pal Singh & Dr. Mahesh Patidar , Faculty of Engineering, Medi-Caps University, Indore, is an authentic work carried out by us.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma. All ideas, data, figures, and text taken from published or unpublished work of other researchers have been duly acknowledged.

Signature: _____

Labdhi Vohra
EN23CS301553

Date: _____

Signature: _____

Lakshita Patel
EN23CS301557

Date: _____

Signature: _____

Lakshita Porwal
EN23CS301558

Date: _____

Certificate

We, **Dr. Madan Pal Singh & Dr. Mahesh Patidar** certify that the project entitled "**Crop Disease Prediction System**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Labdhi Vohra, Lakshita Patel, and Lakshita Porwal** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Madan Pal Singh

Department of Computer
Science & Engineering

Medi-Caps University, Indore

Dr. Mahesh Patidar

Department of Computer
Science & Engineering

Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department
Computer Science & Engineering
Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale.

We also thank **Prof. (Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, Medi-Caps University, for giving us the opportunity to work on this project. We would also like to thank our Head of the Department, **Dr. Kailash Chandra Bandhu**, for his continuous encouragement and valuable guidance throughout the development of this project.

We extend our sincere gratitude to our project guide, **Dr. Madan Pal Singh & Dr. Mahesh Patidar** whose insightful feedback, technical expertise, and constructive suggestions were invaluable throughout every stage of this work. His/her patience and dedication to our academic growth made a significant difference in the quality of this report and the system developed.

We are also thankful to all the faculty members of the Department of Computer Science & Engineering for sharing their knowledge and expertise, which greatly contributed to our understanding of machine learning, web development, and software engineering principles.

We owe special thanks to our families and friends for their moral support, encouragement, and patience throughout the duration of this project. Their unwavering belief in our abilities kept us motivated during challenging times.

Finally, we acknowledge the open-source community, the developers of TensorFlow, Keras, OpenCV, and Flask, and the maintainers of the PlantVillage dataset, whose work formed the technical backbone of this project.

Labdhi Vohra

Lakshita Patel

Lakshita Porwal

B.Tech. III Year, VI Semester

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

Agriculture is one of the most critical sectors in the Indian economy, and crop diseases pose a significant threat to agricultural productivity and food security. Early and accurate identification of plant diseases is essential for minimizing crop losses and ensuring timely treatment. Traditional methods of disease identification rely heavily on the subjective judgment of agricultural experts, which is not always accessible to small-scale or rural farmers.

This project presents the design and development of a **Crop Disease Prediction System**, a web-based application that leverages **Convolutional Neural Networks (CNN)** to automatically detect and classify diseases in crop plants based on uploaded leaf images. The system provides an intelligent, fast, and user-friendly interface through which farmers and agricultural users can upload photographs of affected crop leaves and receive immediate disease diagnosis without requiring specialized expertise.

The proposed system integrates several modern technologies: the **Python** program-ming language serves as the primary development platform, **TensorFlow** and **Keras** are used for building and training the deep learning model, **OpenCV** performs image prepro-cessing operations such as resizing and normalization, and **Flask** is used as the backend web framework. The frontend is developed using **HTML**, **CSS**, and **JavaScript**, while **MySQL** (or a NoSQL alternative) handles data persistence.

The CNN model is trained on the publicly available **PlantVillage** dataset, which contains thousands of labeled leaf images spanning multiple crop types and disease categories. The trained model is capable of classifying diseases in crops such as tomato, potato, pepper, and others with high accuracy. The system processes uploaded images, runs inference through the trained model, and returns the predicted disease name along with appropriate guidance to the user.

Evaluation results demonstrate that the system achieves competitive prediction accuracy while maintaining a response time within the acceptable range of 2-5 seconds. The system is designed to be portable, maintainable, and scalable for future enhancements such as mobile application integration, multi-language support, and real-time treatment recommendations

Keywords: Convolutional Neural Network, Crop Disease Detection, Image Classification, TensorFlow, Keras, OpenCV, Flask, PlantVillage Dataset, Deep Learning, Agricultural Technology

Table of Contents

	Section	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
	Notations & Symbols	1
Chapter 1	Introduction	2
	1.1 Introduction	2
	1.2 Literature Review	3
	1.2.1 Early Image Processing Approaches	3
	1.2.2 Deep Learning and CNN-Based Approaches	3
	1.2.3 Transfer Learning Strategies	3
	1.2.4 Mobile and Real-Time Applications	4
	1.2.5 Web-Based Systems	4
	1.2.6 Identified Research Gap	4
	1.3 Objectives	4
	1.4 Significance of the Study	5
	1.5 Research Design	5
	1.6 Source of Data	6
	1.7 Chapter Scheme	7
Chapter 2	Requirements Specification	8
	2.1 Overview	8
	2.2 User Characteristics	8
	2.2.1 Farmers and General Agricultural Users	8
	2.2.2 Administrators and Developers	9
	2.3 Functional Requirements	9
	2.3.1 User Registration and Authentication	10
	2.3.2 Crop Selection	10
	2.3.3 Image Upload and Validation	10
	2.3.4 Image Processing and Disease Prediction	11
	2.3.5 Result Display	11
	2.4 Dependencies	12
	2.5 Performance Requirements	12
	2.6 Hardware Requirements	13
	2.7 Constraints & Assumptions	14
Chapter 3	Design	16
	3.1 Overview	16
	3.2 Algorithm	16
	3.2.1 Convolutional Neural Network (CNN) Architecture	16
	3.2.2 Image Preprocessing Algorithm	17
	3.3 System Design	18
	3.3.1 Overall System Architecture	18
	3.3.2 Data Flow Diagrams	19

	3.3.2.1 Level 0 DFD - Context Diagram	19
	3.3.2.2 Level 1 DFD	20
	3.3.3 Activity Diagram	21
	3.3.4 Flowchart	21
	3.3.5 Class Diagram	22
	3.3.5.1 Classes	23
	3.3.5.2 Relationships	23
	3.3.6 Entity Relationship (ER) Diagram	24
	3.3.7 Sequence Diagram	25
	3.4 Database Design	26
	3.4.1 Logical Database Design	26
	3.4.1.1 Table: Users	26
	3.4.1.2 Table: Images	27
	3.4.1.3 Table: Predictions	27
	3.4.2 Physical Database Design	27
Chapter 4	Implementation, Testing, and Maintenance	29
	4.1 Introduction to Technologies, IDEs, and Tools	29
	4.1.1 Programming Language: Python	29
	4.1.2 Machine Learning Framework: TensorFlow and Keras	29
	4.1.3 Image Processing Library: OpenCV	29
	4.1.4 Web Framework: Flask	29
	4.1.5 Frontend Technologies	30
	4.1.6 Database: MySQL / NoSQL	30
	4.1.7 Development Tools and IDEs	30
	4.2 Testing Techniques and Test Plans	30
	4.2.1 Unit Testing	31
	4.2.2 Integration Testing	31
	4.2.3 System Testing	31
	4.2.4 Model Accuracy Testing	32
	4.3 Installation Instructions	32
	4.4 End User Instructions	33
Chapter 5	Results and Discussions	34
	5.1 User Interface Representation	34
	5.1.1 Login and Registration Page	34
	5.1.2 Home/Dashboard Page	34
	5.1.3 Image Upload Page	35
	5.1.4 Result Page	35
	5.2 Brief Description of Modules	36
	5.2.1 Authentication Module	36
	5.2.2 Image Handling Module	36
	5.2.3 Image Preprocessing Module	36
	5.2.4 Prediction Module	37
	5.2.5 Database Module	37
	5.3 System Working	37
	5.4 Database Representation	37
Chapter 6	Summary and Conclusions	39
	6.1 Summary	39
	6.2 Conclusions	39
Chapter 7	Future Scope	41
	7.1 Expanded Disease and Crop Coverage	41
	7.2 Mobile Application Development	41
	7.3 Real-Time Treatment Recommendations	41
	7.4 Multi-Language Support	42

	7.5 Integration with IoT Sensors	42
	7.6 Confidence Level Display and Uncertainty Estimation	42
	7.7 Federated Learning	42
	7.8 Integration with Government Agricultural Portals	42
Appendix	Glossary	43
	To Be Determined (TBD) List	45

List of Figures

Figure No.	Figure Title	Page No.
3.1	System Architecture Diagram	19
3.2	Level 0 DFD (Context Diagram)	20
3.3	Level 1 DFD	20
3.4	Activity Diagram	21
3.5	System Flowchart	22
3.6	Class Diagram	24
3.7	Entity Relationship (ER) Diagram	25
3.8	Sequence Diagram	26
5.1	Login and Registration Page	34
5.2	Home/Dashboard Page	35
5.3	Image Upload Page	35
5.4	Disease Prediction Result Page	36

List of Tables

Table No.	Title	Page No.
1.1	Summary of PlantVillage Dataset Used in This Project	7
2.1	Summary of User Classes and Characteristics	9
2.2	System Dependencies	12
2.3	Server Hardware Requirements	13
2.4	Client Hardware Requirements	14
3.1	CNN Architecture Summary	17
3.2	Logical Design – Users Table	26
3.3	Logical Design – Images Table	27
3.4	Logical Design – Predictions Table	27
4.1	Development Tools and Environments	30
4.2	Sample Test Cases	31
5.1	Sample Records – Users Table	37
5.2	Sample Records – Images Table	38
5.3	Sample Records – Predictions Table	38

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
DL	Deep Learning
ER	Entity Relationship
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
ML	Machine Learning
MVC	Model-View-Controller
NoSQL	Not Only Structured Query Language
OOP	Object-Oriented Programming
OS	Operating System
PDF	Portable Document Format
PNG	Portable Network Graphks
RAM	Random Access Memory
ReLU	Rectified Linear Unit
REST	Representational State Transfer
SDS	Software Design Specification
SQL	Structured Query Language
SRS	Software Requirements Specification
TBD	To Be Determined
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Notations & Symbols

Notation	Description
X	Input image matrix
W	Weight matrix of the neural network layer
b	Bias vector
$\sigma(\cdot)$	Activation function (e.g., ReLU, Softmax)
$\hat{y}$	Predicted output label
y	Ground truth label
L	Loss function (Cross-Entropy)
η	Learning rate
σ	Sigmoid activation function
∇	Gradient operator
$*$	Convolution operation
θ	Model parameters
$P(y X)$	Posterior probability of class y given input X
n	Total number of training samples
k	Number of output classes

CHAPTER 1

Introduction

1.1 Introduction

Agriculture has been the backbone of the Indian economy for centuries, supporting the livelihoods of more than half of the country's population. Despite rapid urbanization and industrial growth, the agricultural sector continues to account for a significant share of the national GDP and remains critical to national food security. Within this context, the health of crops is paramount. Any disease affecting crop plants can rapidly escalate from a localized problem to a widespread agricultural catastrophe if not diagnosed and treated in time.

Crop diseases are primarily caused by fungal, bacterial, and viral pathogens, as well as environmental stressors such as nutrient deficiencies, drought, and pest infestations. The visual symptoms of these diseases - manifesting as spots, blights, mosaics, wilts, and discolorations on plant leaves - provide the most accessible means of early identification. Traditionally, this identification process has required the direct involvement of agricultural extension officers, agronomists, or experienced farmers who possess the specialized knowledge to distinguish between different disease types based on visual inspection. Such expertise is not uniformly available, particularly in remote or underdeveloped rural areas where access to agricultural support services is limited.

The rapid advancement of artificial intelligence and deep learning technologies in the past decade has created transformative opportunities across numerous domains, including agriculture. Convolutional Neural Networks (CNNs), a specialized class of deep learning architectures, have demonstrated exceptional capability in image recognition and classification tasks. When applied to the domain of plant pathology, CNNs can be trained to recognize the distinct visual patterns associated with different crop diseases with accuracy levels comparable to, or in many cases exceeding, those of human experts.

The **Crop Disease Prediction System** presented in this report is a web-based application designed to bridge the gap between advanced machine learning technology and the practical needs of farmers and agricultural users. By allowing users to upload photographs of crop leaves through a simple web interface, the system leverages a trained CNN model to analyze the image and predict the disease affecting the plant. The result is communicated in an accessible format, enabling timely intervention without requiring the user to have any technical or agricultural expertise.

This project is not merely a technical exercise; it represents a meaningful application of computer science to a real-world problem that affects millions of people. By democratizing access to plant disease diagnosis, the system has the potential to reduce crop losses, improve farmer incomes, and contribute to broader goals of agricultural sustainability.

1.2 Literature Review

The application of machine learning and image processing techniques to the problem of plant disease detection has been an active area of research since the early 2010s. This section reviews key studies and foundational work that inform the design and implementation of the Crop Disease Prediction System.

1.2.1 Early Image Processing Approaches

Early research in automated plant disease detection relied primarily on traditional image processing techniques such as color histogram analysis, texture feature extraction using Gabor filters, and shape analysis. Phadikar et al. [2] employed a combination of color features and Support Vector Machines (SVMs) to classify rice diseases, achieving moderate accuracy. These approaches, while innovative for their time, were constrained by their dependence on hand-crafted feature extraction, which required significant domain expertise and often failed to generalize across different lighting conditions and image backgrounds.

1.2.2 Deep Learning and CNN-Based Approaches

The landmark study by Mohanty et al. [1] published in 2016 fundamentally transformed the field. Using the PlantVillage dataset - which contains over 54,000 labeled photographs of diseased and healthy plant leaves - the authors trained a series of deep CNN architectures, including AlexNet and GoogLeNet, achieving a top-5 accuracy exceeding 99% under controlled conditions. This study demonstrated the enormous potential of deep learning for large-scale plant disease identification and established the PlantVillage dataset as the standard benchmark for subsequent research.

1.2.3 Transfer Learning Strategies

Subsequent research explored the use of transfer learning - adapting pre-trained models such as VGG16, ResNet-50, and InceptionV3 - to reduce training time and improve performance on smaller datasets. Ramcharan et al. [3] applied transfer learning with GoogLeNet to detect cassava diseases from field-collected images, achieving accuracy above 93%. Transfer learning proved particularly effective in scenarios where the available labeled data was insufficient for training a model from scratch.

1.2.4 Mobile and Real-Time Applications

With the proliferation of smartphones, researchers have increasingly focused on developing lightweight, mobile-compatible disease detection models. MobileNet and EfficientNet architectures have been widely adopted for this purpose due to their compact size and efficient inference capabilities. Several mobile applications have been developed and deployed in agricultural settings, though challenges related to image quality variation, disease co-occurrence, and environmental noise remain significant hurdles.

1.2.5 Web-Based Systems

More recently, researchers have developed web-based platforms that integrate trained CNN models with accessible user interfaces. These platforms allow farmers to upload images from any internet-connected device and receive disease predictions in near real-time. The work of Kamilaris and Prenafeta-Boldu [4] provides a comprehensive review of deep learning applications in agriculture, highlighting both the progress made and the challenges that remain, including the need for datasets that reflect real-world field conditions rather than controlled laboratory environments.

1.2.6 Identified Research Gap

Despite the extensive research in this area, there remains a significant gap in the availability of practical, easy-to-use, web-based tools that are accessible to non-technical users - particularly farmers in developing countries. Most research prototypes require technical expertise to set up and operate, and few have been designed with the specific constraints and needs of rural users in mind. The Crop Disease Prediction System presented in this report directly addresses this gap by providing a simple, intuitive web interface backed by a robust CNN model.

1.3 Objectives

The primary objectives of the Crop Disease Prediction System are as follows:

1. To develop a web-based application that enables farmers and agricultural users to identify crop diseases by uploading leaf images through a simple interface.
2. To design and train a Convolutional Neural Network (CNN) model capable of accurately classifying multiple crop disease categories using the PlantVillage dataset.
3. To integrate image preprocessing techniques using the OpenCV library to enhance the quality and consistency of input images prior to model inference.
4. To implement a secure user authentication system with registration and login functionality to manage access to the prediction services.

5. To provide users with clear, interpretable disease prediction results, including the disease name and, where applicable, a confidence level.
6. To build a maintainable and scalable system architecture that supports future enhancements, including mobile deployment and multi-language support.
7. To evaluate the performance of the system in terms of prediction accuracy, response time, and usability.

1.4 Significance of the Study

The significance of the Crop Disease Prediction System extends across multiple dimensions - technological, agricultural, economic, and social.

Technological Significance: This project demonstrates a practical application of state-of-the-art deep learning techniques to a problem of national and global importance. The integration of CNN-based image classification with a web-based delivery platform represents a successful case study in applied artificial intelligence.

Agricultural Significance: The system provides farmers with an accessible tool for early disease detection, which is crucial for preventing the spread of disease and minimizing crop losses. Early identification allows for timely application of appropriate treatments, reducing the reliance on broad-spectrum pesticides and thereby contributing to more sustainable agricultural practices.

Economic Significance: Crop diseases cause billions of dollars in agricultural losses annually. By enabling faster and more accurate disease diagnosis, the system has the potential to significantly reduce these losses, thereby improving farmer incomes and overall agricultural productivity.

Social Significance: The system is designed to be usable by individuals with minimal technical literacy, making advanced agricultural technology accessible to a broader segment of the population. This democratization of agricultural AI has the potential to reduce the information asymmetry between resource-rich and resource-poor farmers.

1.5 Research Design

The research methodology adopted for this project follows a structured software development lifecycle (SDLC) approach. The development process was carried out in the following phases:

Phase 1: Requirements Analysis - Identification of functional and non-functional system requirements through a review of existing literature, domain analysis, and consultation with potential users. The output of this phase is documented in the Software

Requirements Specification (SRS).

Phase 2: System Design - Design of the overall system architecture, user interface layouts, database schema, and machine learning model architecture. The output is documented in the Software Design Specification (SDS).

Phase 3: Dataset Preparation - Acquisition and preprocessing of the PlantVillage dataset, including image resizing, normalization, data augmentation, and train-test splitting.

Phase 4: Model Development - Design, training, validation, and optimization of the CNN model using TensorFlow and Keras.

Phase 5: Application Development- Implementation of the frontend user interface and the Flask backend, including the integration of the trained model into the prediction pipeline.

Phase 6: Testing and Evaluation - Unit testing, integration testing, and system testing to verify functionality, performance, and usability.

Phase 7: Deployment and Documentation - Preparation of the deployable application and the accompanying project report.

1.6 Source of Data

The primary data source for training and validating the CNN model is the **PlantVillage Dataset**, which is one of the largest and most widely used publicly available datasets for plant disease research. Key characteristics of the dataset are as follows:

- The dataset contains approximately **54,306 images** of healthy and diseased plant leaves.
- Images are organized into **38 classes**, corresponding to different combinations of plant species and disease types.
- The dataset covers **14 crop species**, including tomato, potato, pepper, apple, corn, grape, and several others.
- Each class contains color images of leaves captured under controlled laboratory conditions against a uniform background.
- The dataset is available on the Kaggle platform and through the official PlantVillage repository.

For the purposes of this project, a representative subset of the full dataset was selected to train the model, focusing on a set of commonly cultivated crops and their associated diseases. The dataset was split into training (80%), validation (10%), and testing (10%)

subsets to enable proper model evaluation.

Table 1.1: Summary of PlantVillage Dataset Used in This Project

Crop Type	Disease Categories	Approx. Images
Tomato	10 (including healthy)	18,160
Potato	3 (including healthy)	2,152
Pepper	2 (including healthy)	2,475
Apple	4 (including healthy)	3,171
Com	4 (including healthy)	3,852
Grape	4 (including healthy)	4,063

1.7 Chapter Scheme

The remainder of this report is organized as follows:

Chapter 2 - Requirements Specification: This chapter provides a detailed description of the functional and non-functional requirements of the system, including user characteristics, hardware and software constraints, performance requirements, and assumptions.

Chapter 3 - Design: This chapter presents the complete system design, including the CNN algorithm, system architecture, Data Flow Diagrams (DFD) at Level 0 and Level 1, the Activity Diagram, Flowchart, Class Diagram, Entity Relationship (ER) Diagram, Sequence Diagram, and the logical and physical database design.

Chapter 4 - Implementation, Testing, and Maintenance: This chapter describes the technologies, tools, and IDEs used for implementation, followed by a discussion of the testing strategy, test cases, installation instructions, and end-user guide.

Chapter 5 - Results and Discussions: This chapter presents the user interface of the developed system, provides a description of each functional module, and includes snapshots of key system screens and database tables with accompanying explanations.

Chapter 6 - Summary and Conclusions: This chapter summarizes the work carried out, highlights the key contributions of the project, and draws conclusions based on the results obtained.

Chapter 7 - Future Scope: This chapter identifies potential directions for future work and enhancements to the system, including mobile deployment, multi-language support, expanded disease coverage, and integration with IoT sensors.

CHAPTER 2

Requirements Specification

2.1 Overview

This chapter provides a comprehensive description of the requirements for the Crop Disease Prediction System, drawing primarily from the Software Requirements Specification (SRS) document prepared for this project. The requirements are organized according to user characteristics, functional requirements, system dependencies, performance requirements, hardware requirements, and system constraints and assumptions.

A well-defined requirements specification is a critical foundation for any software development project. It ensures that all stakeholders - including developers, testers, project supervisors, and end users - have a shared understanding of what the system is intended to do, under what constraints it must operate, and what quality standards it must meet. The requirements presented in this chapter were derived through a careful analysis of the problem domain, a review of related systems, and a consideration of the practical needs of the intended user base.

2.2 User Characteristics

The Crop Disease Prediction System is intended for two primary categories of users, each with distinct characteristics, needs, and levels of technical expertise.

2.2.1 Farmers and General Agricultural Users

The largest and most important user group consists of farmers, agricultural workers, and general users who wish to identify diseases affecting their crops. These users represent the primary beneficiaries of the system and have the following characteristics:

- They typically possess **basic to moderate digital literacy**, sufficient to operate a smartphone or desktop computer and navigate a web browser.
- They are capable of **capturing or uploading photographs** of crop leaves, either using a smartphone camera or by transferring images from other devices.
- They may have **limited or no knowledge** of agricultural science or plant pathology beyond practical farming experience.
- They are primarily motivated by a desire to **quickly and accurately identify** the

disease affecting their crops so that they can take appropriate action to prevent further spread and economic loss.

- They access the system through **web browsers** on a variety of devices, including desktop computers, laptops, tablets, and smartphones.

The system must therefore prioritize simplicity, clarity, and accessibility in its user interface design to accommodate this diverse and largely non-technical user base.

2.2.2 Administrators and Developers

The second user group consists of system administrators and software developers who are responsible for maintaining, updating, and extending the system. These users have the following characteristics:

- They possess **advanced technical knowledge** of web development, machine learning, and database management.
- They are responsible for **monitoring system performance**, managing the database, and ensuring the availability and reliability of the application.
- They are responsible for **updating the machine learning model** when new disease categories are added or when improved model versions become available.
- They may occasionally need to **diagnose and resolve bugs or performance issues** in the system.

Table 2.1: Summary of User Classes and Characteristics

User Class	Characteristics	Primary Activities
Farmer/ General User	Basic digital literacy; non-technical agricultural background	Register, login, upload image, view prediction result
Administrator Developer	Advanced technical knowledge; software and ML expertise	Maintain system, update model, manage database, monitor performance

2.3 Functional Requirements

Functional requirements specify what the system must do - that is, the specific behaviors and functions that it must provide. The functional requirements of the Crop Disease Prediction System are organized by feature category.

2.3.1 User Registration and Authentication (REQ-AUTH)

- REQ-AUTH-1:** The system shall provide a user registration interface that allows new users to create an account by providing their name, email address, and a password.
- REQ-AUTH-2:** The system shall validate the registration form inputs, ensuring that all required fields are completed, that the email address is in a valid format, and that the password meets minimum security requirements.
- REQ-AUTH-3:** The system shall store user credentials securely in the database, with passwords stored as hashed values using an appropriate cryptographic hash function.
- REQ-AUTH-4:** The system shall provide a login interface that allows registered users to authenticate themselves using their email address and password.
- REQ-AUTH-5:** The system shall verify user credentials against the stored values and grant access to the prediction services upon successful authentication.
- REQ-AUTH-6:** The system shall display a clear and informative error message if the user provides incorrect login credentials.
- REQ-AUTH-7:** The system shall support user session management, maintaining the authenticated state across multiple pages during a single browser session.
- REQ-AUTH-8:** The system shall provide a logout function that terminates the user session and returns the user to the login page.

2.3.2 Crop Selection (REQ-CROP)

- REQ-CROP-1:** The system shall provide a crop selection interface that presents the user with a list of supported crop types for which disease prediction is available.
- REQ-CROP-2:** The system shall allow the user to select exactly one crop type before proceeding to the image upload step.
- REQ-CROP-3:** The system shall display appropriate guidance to the user regarding the correct type of leaf image to capture for the selected crop.
- REQ-CROP-4:** The system shall update the available prediction categories based on the crop type selected by the user.

2.3.3 Image Upload and Validation (REQ-IMG)

- REQ-IMG-1:** The system shall provide an image upload interface that allows the user to select and upload a photograph of a crop leaf from their local device.

- REQ-IMG-2:** The system shall accept image files in the following formats: JPEG (JPG) and PNG.
- REQ-IMG-3:** The system shall validate the format of the uploaded file and display an appropriate error message if the file format is not supported.
- REQ-IMG-4:** The system shall validate the size of the uploaded file and reject files that exceed a predefined maximum size limit (to be determined based on server capacity constraints).
- REQ-IMG-5:** The system shall perform a basic quality check on the uploaded image to verify that it is suitable for processing.
- REQ-IMG-6:** The system shall display a preview of the uploaded image to the user before processing, allowing the user to confirm or replace the image.
- REQ-IMG-7:** The system shall handle cases where no file is uploaded and prompt the user to select an image before proceeding.

2.3.4 Image Processing and Disease Prediction (REQ-PRED)

- REQ-PRED-1:** The system shall automatically preprocess the uploaded image upon submission, including operations such as resizing to the model's required input dimensions, normalization of pixel values, and noise reduction.
- REQ-PRED-2:** The system shall pass the preprocessed image to the trained CNN model for inference.
- REQ-PRED-3:** The system shall generate a disease prediction for the submitted image, identifying the specific disease or confirming that the plant is healthy.
- REQ-PRED-4:** The system shall handle exceptions and errors that arise during image processing or model inference gracefully, displaying appropriate user-facing error messages without exposing internal technical details.
- REQ-PRED-5:** The system shall complete the prediction process and return a result to the user within a maximum response time of 5 seconds under normal operating conditions.

2.3.5 Result Display (REQ-RESULT)

- REQ-RESULT-1:** The system shall display the predicted disease name in a clear and prominently positioned area of the result page.
- REQ-RESULT-2:** The system shall display a confidence level or probability score associated with the prediction (to be determined).

REQ-RESULT-3: The system shall include a disclaimer on the result page advising users that the prediction result is provided as a suggestion only and should be verified by a qualified agricultural expert before taking action.

REQ-RESULT-4: The system shall provide the user with the option to upload another image for a new prediction after viewing the result.

REQ-RESULT-5: The system shall store the prediction result in the database for the purposes of historical tracking (if this feature is enabled).

2.4 Dependencies

The Crop Disease Prediction System depends on the following external resources, libraries, and frameworks for its correct operation:

Table 2.2: System Dependencies

Dependency	Type	Role in System
PlantVillage Dataset	Dataset	Primary training and validation data source for the CNN model
TensorFlow 2.x	ML Library	Core framework for building and running the deep learning model
Keras	MLAPI	High-level API for CNN model design and training
OpenCV	Image Library	Image preprocessing (resize, normalize, filter)
Flask	Web Framework	Backend server for handling HTTP requests and routing
NoSQL	Database	Storage of user credentials and prediction history
Python 3.x	Language	Primary programming language for backend and ML components
HTML/CSS/JS	Frontend Tech	User interface rendering in web browsers

2.5 Performance Requirements

The following performance requirements have been established for the Crop Disease Prediction System:

PERF-1: Response Time: The system shall generate and display a disease prediction result within a maximum of **2 to 5 seconds** from the time the user submits the image for processing, under standard network and hardware conditions.

PERF-2: Concurrency: The system shall be capable of handling multiple simultaneous user sessions without significant degradation in response time. The minimum acceptable concurrent user load shall be defined based on the deployment environment.

PERF-3: Availability: The system shall aim for a minimum uptime of **99%** during the operational period, with planned maintenance windows communicated in advance.

PERF-4: Scalability: The system architecture shall be designed to support scaling of the backend services to accommodate growth in user load without requiring fundamental redesign.

PERF-5: Model Accuracy: The CNN model shall achieve a minimum classification accuracy of **85%** on the test dataset. The target accuracy is above **90%** for the primary crop categories.

2.6 Hardware Requirements

The hardware requirements for the Crop Disease Prediction System are defined for two contexts: the development/server environment and the end-user (client) environment.

2.6.1 Server/ Development Environment

Table 2.3: Server Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 or equivalent (multi-core)
RAM	8 GB (16 GB recommended for model training)
Storage	50 GB available disk space
GPU	NVIDIA GPU with CUDA support (recommended for training)
Network	Broadband internet connection

2.6.2 Client/ End-User Environment

Table 2.4: Client Hardware Requirements

Component	Minimum Specification
Device	Desktop computer, laptop, tablet, or smartphone
Processor	Any modern processor (mobile or desktop)
RAM	2GB
Camera	For capturing crop leaf images (optional; images may be pre-captured)
Network	Internet connection (Wi-Fi or mobile data)
Browser	Google Chrome (v80+), Mozilla Firefox (v75+), or Microsoft Edge (v80+)

2.7 Constraints and Assumptions

2.7.1 Design and Implementation Constraints

- CON-1:** The CNN model requires a sufficiently large and labeled dataset for training. The quality and diversity of the training data directly affect the prediction accuracy of the system.
- CON-2:** The quality, resolution, and clarity of the input image significantly affect the prediction accuracy. Images captured under poor lighting conditions, with blurred focus, or with significant background clutter may result in reduced prediction reliability.
- CON-3:** The system is implemented exclusively in the **Python** programming language, using the TensorFlow and Keras deep learning frameworks. This limits the pool of potential developers who can maintain and extend the system.
- CON-4:** The initial version of the system is designed as a **web-based application** and does not include native mobile application support, although this is planned for future development.
- CON-5:** An active **internet connection** is required for users to access the application if it is deployed on a remote server. Offline functionality is not supported **in** the current version.

2.7.2 Assumptions

- ASS-1:** It is assumed that users have access to a device with an internet connection and a web browser capable of running modern web applications.
- ASS-2:** It is assumed that users are able to capture or upload reasonably clear photographs

of crop leaves, with the leaf and its symptoms visible and in focus.

ASS-3: It is assumed that the uploaded images contain visible symptoms of the relevant disease. Images of healthy plants may return a "Healthy" prediction, but images that are too heavily damaged, decomposed, or otherwise unrelated to plant leaves may produce unreliable results.

ASS-4: It is assumed that the PlantVillage dataset is representative of the diseases and visual characteristics encountered in the field, and that the trained model will generalize adequately to real-world images.

ASS-5: It is assumed that the system will operate on a standard server environment with Python and Flask installed.

CHAPTER 3

Design

3.1 Overview

The design of the Crop Disease Prediction System encompasses multiple levels of abstraction, from the high-level system architecture down to the detailed logic of individual components. This chapter presents the complete design of the system, including the CNN algorithm, system architecture, data flow diagrams, activity diagram, flowchart, class diagram, entity relationship diagram, sequence diagram, and database design. Together, these artifacts provide a comprehensive blueprint for the implementation of the system.

3.2 Algorithm

3.2.1 Convolutional Neural Network (CNN) Architecture

The core algorithmic component of the Crop Disease Prediction System is a Convolutional Neural Network (CNN), a class of deep learning architecture particularly well-suited to image classification tasks. CNNs are designed to automatically and adaptively learn spatial hierarchies of features from input images, progressing from low-level features such as edges and textures at earlier layers to high-level semantic features representing disease-specific patterns at deeper layers.

The CNN model designed for this project follows a sequential architecture consisting of the following layer types:

1. **Input Layer:** Accepts an image of dimensions $224 \times 224 \times 3$ (height $\times$ width $\times$ color channels).
2. **Convolutional Layers:** Each convolutional layer applies a set of learnable filters of size 3×3 to the input feature map, producing a set of activation maps that highlight the presence of specific features.
3. **Batch Normalization Layers:** Applied after each convolutional layer to normalize the activations, accelerating training and improving generalization.
4. **Activation Function (ReLU):** The Rectified Linear Unit (ReLU) activation function, defined as $J(x) = \max(0, x)$, is applied element-wise to introduce non-linearity into the model.

5. **Pooling Layers:** Max pooling layers with a pool size of 2×2 are applied to progressively reduce the spatial dimensions of the feature maps, reducing computational cost and providing translation invariance.
6. **Dropout Layers:** Dropout regularization (with a rate of 0.5) is applied to the fully connected layers to prevent overfitting by randomly deactivating a fraction of neurons during training.
7. **Fully Connected (Dense) Layers:** Following the convolutional and pooling layers, the feature maps are flattened and passed through one or more fully connected layers for high-level reasoning.
8. **Output Layer:** A Softmax activation function is applied to produce a probability distribution over the k disease classes, where k is the number of classes in the training dataset.

Table 3.1: CNN Architecture Summary

Layer Type	Filters/Units	Kernel/Pool Size	Activation
Conv2D BatchNorm MaxPooling2D	32 - -	3x3 2x2	ReLU
Conv2D BatchNorm MaxPooling2D	64 - -	3x3 2x2	ReLU
Conv2D BatchNorm MaxPooling2D Flatten	128 - -	3x3 2x2	ReLU
Dense Dropout (0.5)	512		ReLU
Dense (Output)	k		Softmax

The model is compiled with the **Adam** optimizer and a **categorical cross-entropy** loss function. Training is performed over a defined number of epochs with early stopping based on validation loss to prevent overfitting.

3.2.2 Image Preprocessing Algorithm

Prior to feeding images into the CNN model, each uploaded image undergoes the following preprocessing pipeline implemented using OpenCV:

1. **Read Image:** The image file is read from the upload directory in BGR color format.

2. **Resize:** The image is resized to 224 x 224 pixels using bilinear interpolation to match the model's expected input dimensions.
3. **Convert Color Space:** The image is converted from BGR to RGB color space, as required by the model.
4. **Normalize Pixel Values:** Pixel values are scaled from the range [0, 255] to [0, 1] by dividing by 255.
5. **Expand Dimensions:** A batch dimension is added, transforming the image array from shape (224,224, 3) to (1,224,224, 3), as required by the Keras model's predict function.

3.3 System Design

3.3.1 Overall System Architecture

The Crop Disease Prediction System follows a **Client-Server Architecture** organized into four distinct layers: the Presentation Layer, the Application Layer, the Machine Learning Layer, and the Data Layer.

- **Presentation Layer (Frontend):** Developed using HTML, CSS, and JavaScript, this layer provides the user interface through which users interact with the system. It handles user inputs, displays forms for image upload, and renders prediction results.
- **Application Layer (Backend):** Implemented using the Flask web framework in Python, this layer processes HTTP requests from the frontend, orchestrates the image preprocessing and prediction pipeline, and returns responses to the client.
- **Machine Learning Layer:** The trained CNN model, serialized in the HDF5 format (.h5), is loaded by the backend and used to perform inference on preprocessed images.
- **Data Layer:** A relational database (MySQL) or a NoSQL database stores user registration data, image metadata, and prediction history.

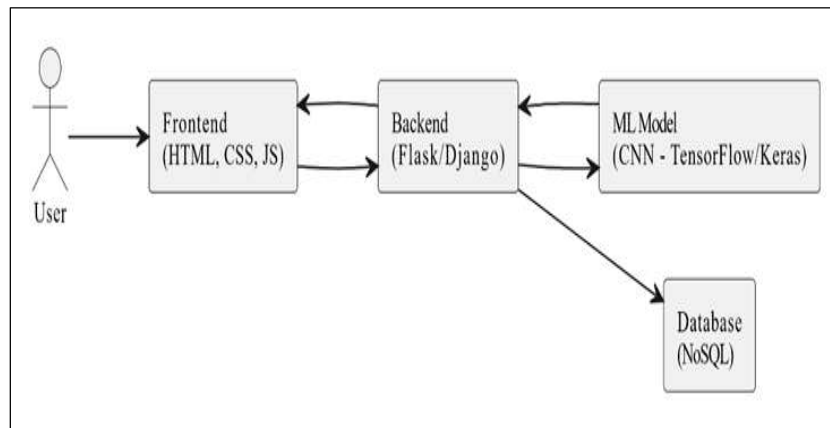


Figure 3.1: System Architecture Diagram

3.3.2 Data Flow Diagrams

3.3.2.1 Level O DFD- Context Diagram

The Level O Data Flow Diagram (also known as the Context Diagram) provides the highest-level view of the Crop Disease Prediction System. It represents the entire system as a single process and shows the external entities that interact with it, along with the primary data flows.

External Entities:

- **User (Farmer):** Provides crop leaf images and authentication credentials; receives disease prediction results.
- **Administrator:** Provides model updates and system configuration; accesses system logs and monitoring data.

Primary Data Flows:

- User → System: Login credentials, crop selection, uploaded leaf image.
- System → User: Authentication response, prediction result.
- Administrator → System: Model updates, configuration changes.
- System → Administrator: System logs, performance metrics.

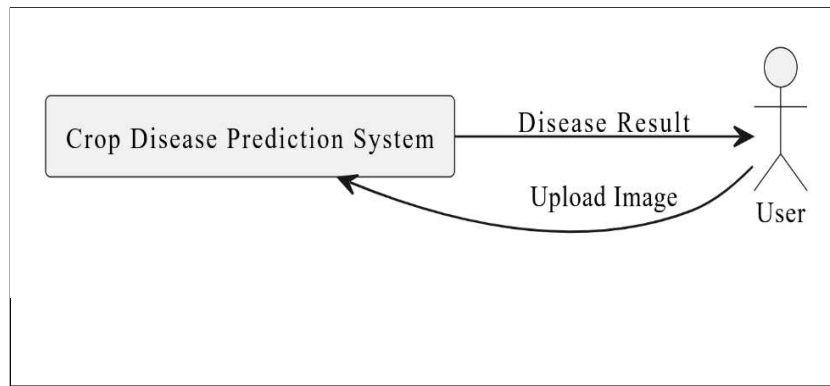


Figure 3.2: Level 0 DFD (Context Diagram)

3.3.2.2 Level 1 DFD

The Level 1 DFD decomposes the single top-level process of the Context Diagram into four sub-processes, revealing the internal data flows between them.

Sub-processes:

1. **Process 1 - User Authentication:** Receives login/registration credentials from the user, validates them against the database, and grants or denies access.
2. **Process 2 - Image Upload and Validation:** Receives the uploaded leaf image from the user, validates its format and quality, and passes it to the image processing module.
3. **Process 3 - Image Processing and Prediction:** Preprocesses the image using OpenCV and passes it to the CNN model for disease classification.
4. **Process 4 - Result Generation and Display:** Receives the model's prediction output, formats it for display, optionally stores it in the database, and sends the result to the user.

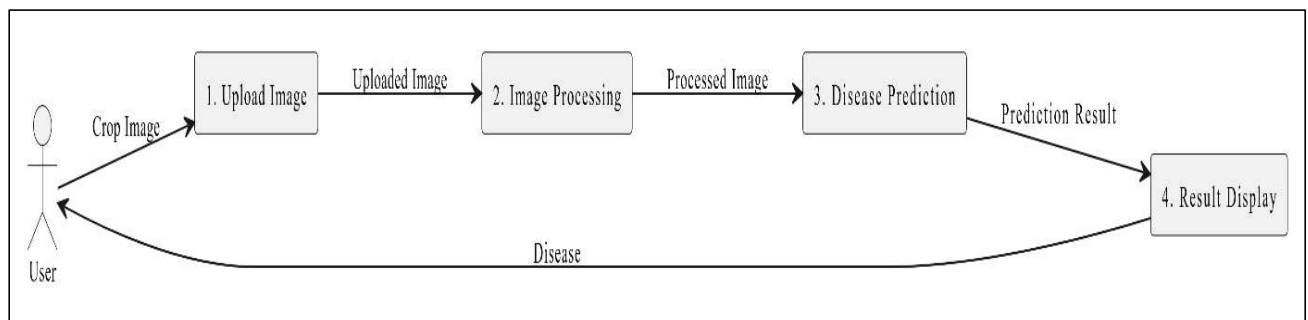


Figure 3.3: Level 1 DFD

3.3.3 Activity Diagram

The Activity Diagram captures the dynamic workflow of the Crop Disease Prediction System, illustrating the sequence of activities that occur from the moment a user initiates a session to the receipt of a disease prediction result.

The workflow begins when the user accesses the system through the web interface and logs in or registers using their credentials. After successful authentication, the user is redirected to the crop selection page, where they choose the desired crop type for disease detection. The user then moves to the image upload page and selects a leaf image. The system checks the uploaded image for validity; if it is invalid or unsupported, an error message is displayed and the user is asked to upload another image. If the image is valid, it undergoes preprocessing and is passed to the CNN model for analysis.

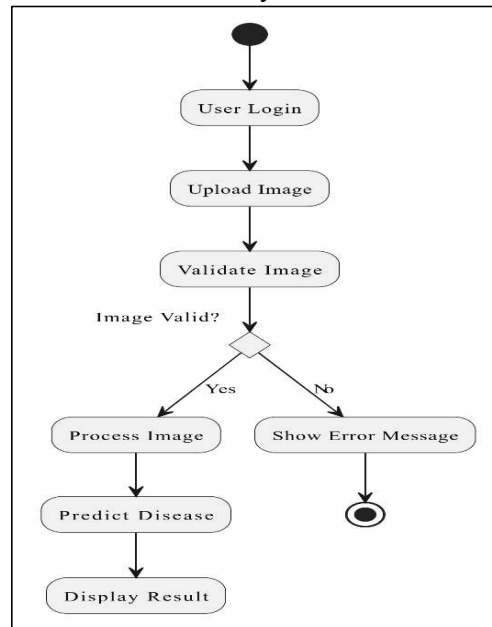


Figure 3.4: Activity Diagram

3.3.4 Flowchart

The system flowchart provides a step-by-step algorithmic representation of the main prediction workflow:

1. **START**
2. User accesses the web application and logs in.
3. User selects the crop type from the available options.
4. User uploads a leaf image.
5. **Decision:** Is the image format valid (JPG/PNG)?
 - No: Display error message. Return to Step 4.

- Yes: Proceed to Step 6.
- 6. **Decision:** Is the image quality acceptable?
 - No: Display quality warning. Return to Step 4.
 - Yes: Proceed to Step 7.
- 7. Preprocess the image (resize, normalize, expand dimensions).
- 8. Feed the preprocessed image to the CNN model.
- 9. **Decision:** Did the model inference succeed?
 - No: Display internal error message. Notify administrator.
 - Yes: Proceed to Step 10.
- 10. Display the predicted disease name and confidence level to the user.
- 11. Optionally store the prediction record in the database.
- 12. **Decision:** Does the user wish to perform another prediction?
 - Yes: Return to Step 3.
 - No: Proceed to Step 13.
- 13. **END**

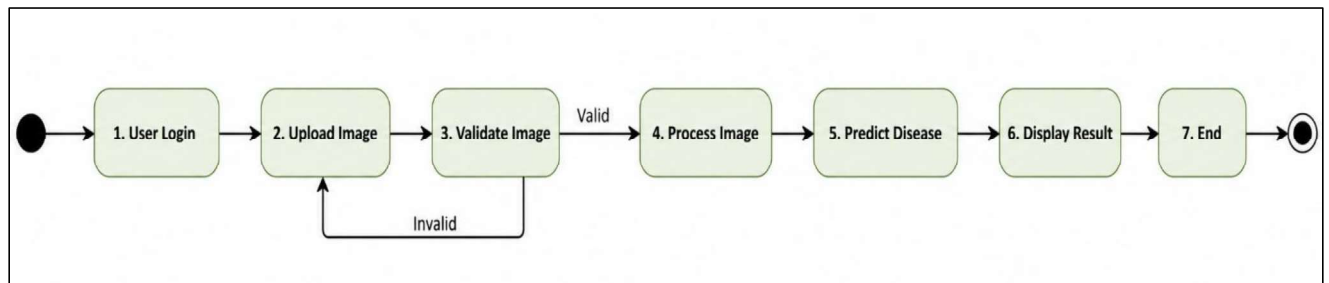


Figure 3.5: System Flowchart

3.3.5 Class Diagram

The Class Diagram represents the object-oriented structure of the Crop Disease Prediction System, showing the main classes, their attributes, methods, and the relationships between them.

3.3.5.1 Classes

User Class

- Attributes: user_id (int), name (str), email (str), password_hash (str), created_at (datetime)
- Methods: register(), login(), logout(), update_profile()

Image Class

- Attributes: image_id (int), user_id (int), file_path (str), upload_date (datetime), format (str)
- Methods: upload(), validate(), preprocess(), delete()

Prediction Class

- Attributes: prediction_id (int), image_id (int), predicted_disease (str), confidence (float), prediction_date (datetime)
- Methods: generate(), display(), save_to_db()

CNNModel Class

- Attributes: model_path (str), model (object), class_labels (list)
- Methods: load_model(), preprocess_image(), predict(image), get_class_name(index)

3.3.5.2 Relationships

- **User** has a one-to-many relationship with **Image** (a user can upload multiple images).
- **Image** has a one-to-one relationship with **Prediction** (each uploaded image generates one prediction).
- **CNNModel** is used by **Prediction** (the prediction class invokes the CNN model to generate predictions).

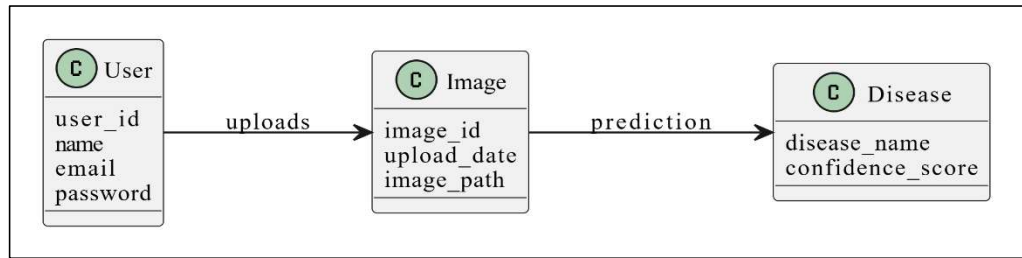


Figure 3.6: Class Diagram

3.3.6 Entity Relationship (ER) Diagram

The Entity Relationship (ER) Model represents the logical structure of the data managed by the Crop Disease Prediction System. The system's database design is built around three primary entities and their relationships.

Entities and Attributes:

- **User:** (useLid [PK], name, email, password_hash, created_at)
- **Image:** (image_id [PK], user_id [FK], file_path, upload_date, format)
- **Prediction:** (prediction_id [PK], image_id [FK], predicted_disease, confidence, prediction_date)

Relationships:

- **UPLOADS:** A **User** uploads one or more **Images**. Cardinality: 1 (User) to N (Images).
- **GENERATES:** An **Image** generates exactly one **Prediction**. Cardinality: 1 (Image) to 1 (Prediction).

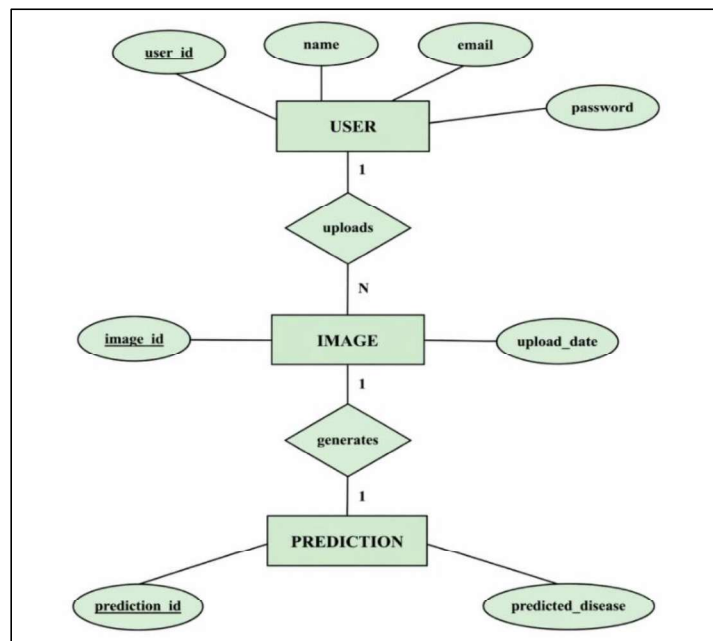


Figure 3.7: Entity Relationship (ER) Diagram

3.3.7 Sequence Diagram

The Sequence Diagram illustrates the time-ordered interactions between the different objects and components of the system during the core use case of image upload and disease prediction.

Objects/Participants:

1. User
2. Frontend (Web Browser)
3. Backend (Flask Server)
4. CNNModel
5. Database

Sequence of Interactions:

1. **User** → **Frontend** User enters login credentials.
2. **Frontend** → **Backend** Sends login request (POST /login).
3. **Backend** → **Database** Queries user details using email.
4. **Database** → **Backend** Returns user record.
5. **Backend** → **Frontend** Sends authentication success/failure response.
6. **User** → **Frontend** User selects crop and uploads image.
7. **Frontend** → **Backend** Sends prediction request (POST /predict with image file).
8. **Backend** Validates image format and quality.
9. **Backend** → **CNN Model** Preprocesses the image and sends it to the CNN model.
10. **CNN Model** → **Backend** Returns disease prediction result.
11. **Backend** → **Frontend** Sends prediction result.
12. **Frontend** → **User** Displays result to the user.

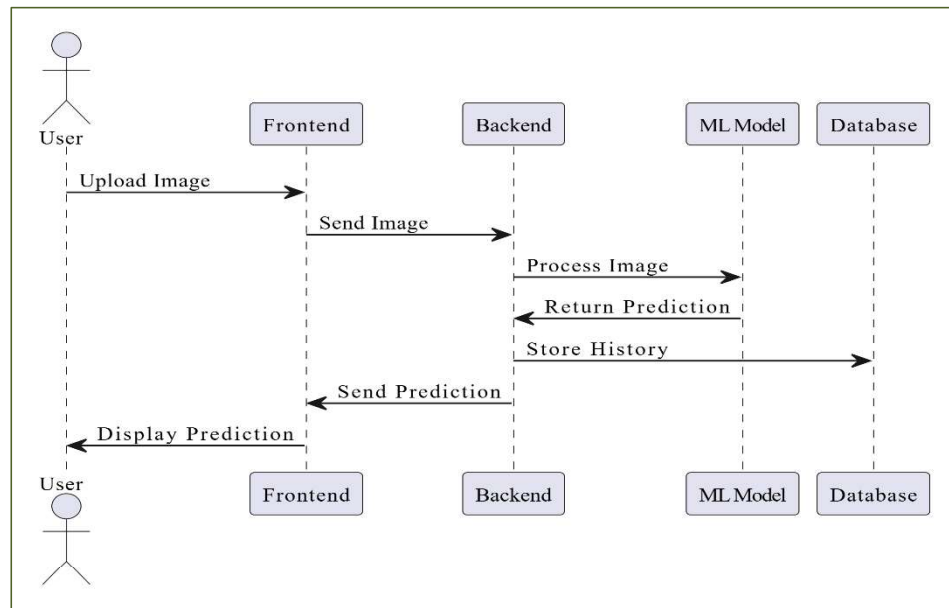


Figure 3.8: Sequence Diagram

3.4 Database Design

3.4.1 Logical Database Design

The logical database design defines the structure of data stored in the system using a NoSQL approach. Instead of tables, the system uses **collections and documents**, which provide flexibility and scalability. The system mainly consists of three collections: **Users, Images, and Predictions**.

3.4.1.1 Table: Users

Table 3.2: Logical Design - Users Table

Field Name	Data Type	Description
user_id	ObjectId	Unique user identifier
name	String	Full name of the user
email	String	User's email (unique)
password	String	Hashed password
created_at	Date	Registration timestamp

3.4.1.2 Table: Images

Table 3.3: Logical Design - Images Table

Field Name	Data Type	Description
image_id	ObjectId	Unique image identifier
user_id	ObjectId	Reference to user
file_path	String	Path of stored image
upload_date	Date	Upload timestamp
format	String	Image format (JPG/PNG)

3.4.1.3 Table: Predictions

Table 3.4: Logical Design - Predictions Table

Field Name	Data Type	Description
prediction_id	ObjectId	Unique prediction identifier
image_id	ObjectId	Reference to image
predicted_disease	String	Predicted disease name
confidence	Number	Prediction confidence
prediction_date	Date	Prediction timestamp

3.4.2 Physical Database Design

The physical database design translates the logical model into specifications suitable for implementation in a specific database management system. For this project, a NoSQL database such as MongoDB is used to provide flexible and scalable storage for prediction history. The SQL schema for the primary tables is as follows:

User Collection

```
{
  "_id": "ObjectId",
  "name": "Shri Sharma",
  "email": "Shri@email.com",
  "password": "hashed_password",
  "created_at": "2026-04-29"
}
```

Image Collection

```
{
  "_id": "ObjectId",
  "user_id": "ObjectId",
  "file_path": "/uploads/leaf1.jpg",
  "upload_date": "2026-04-29",
  "format": "JPG"
}
```


Prediction Collections

```
{  
  "_id": "ObjectId",  
  "image_id": "ObjectId",  
  "predicted_disease": "Leaf Blight",  
  "confidence": 0.92,  
  "prediction_date": "2026-04-29"  
}
```

Indexing strategies are applied to frequently queried fields to improve performance. A unique index is created on the email field in the Users collection, and indexes are applied on user_id in the Images collection and image_id in the Predictions collection to enable faster data retrieval.

CHAPTER 4

Implementation, Testing, and Maintenance

4.1 Introduction to Technologies, IDEs, and Tools

4.1.1 Programming Language: Python

Python (version 3.8 or higher) serves as the primary programming language for all backend and machine learning components of the Crop Disease Prediction System. Python was chosen for several compelling reasons. First, it offers an extensive ecosystem of libraries specifically designed for machine learning and data science, including TensorFlow, Keras, NumPy, and OpenCV. Second, Python's clean and readable syntax reduces development time and cognitive overhead. Third, the language enjoys widespread adoption in the research and development community, making it easier to find support, documentation, and pre-trained models.

4.1.2 Machine Learning Framework: TensorFlow and Keras

TensorFlow (version 2.x) is the core machine learning framework used to build, train, and deploy the CNN model. TensorFlow provides a comprehensive set of tools for numerical computation and machine learning, with efficient support for GPU-accelerated training. **Keras**, integrated as the high-level API of TensorFlow 2.x (tf.keras), provides an intuitive and user-friendly interface for designing and training neural networks. The Keras Sequential API is used to stack the layers of the CNN model, while the Functional API is available for more complex architectures if required.

4.1.3 Image Processing Library: OpenCV

OpenCV (Open Source Computer Vision Library) is used for all image preprocessing operations in the system. Specifically, OpenCV handles the reading of uploaded image files, resizing to the model's required input dimensions, color space conversion, and pixel normalization. OpenCV was chosen for its high performance, extensive documentation, and seamless integration with NumPy arrays, which are the standard data format used by Keras.

4.1.4 Web Framework: Flask

Flask is a lightweight and flexible Python web framework used to build the backend of the application. Flask follows the WSGI (Web Server Gateway Interface) standard and

supports the rapid development of web applications without imposing a rigid project structure. Key Flask features used in this project include routing (URL endpoint mapping), request handling, file upload management via `request.files`, and HTML template rendering via Jinja2. Flask's simplicity and low overhead make it an ideal choice for a project of this scale.

4.1.5 Frontend Technologies

The frontend user interface is built using standard web technologies:

- **HTMLS:** Provides the structural markup of all web pages in the application.
- **CSS3:** Provides visual styling, layout, and responsive design.
- **JavaScript (ES6):** Handles client-side interactivity, form validation, and asynchronous communication with the backend.

4.1.6 Database: NoSQL

A NoSQL database such as MongoDB is used as the primary database system for storing user data and prediction history. It provides flexibility in handling semi-structured data and supports scalable, schema-less storage suitable for this application.

4.1.7 Development Tools and IDEs

Table 4.1: Development Tools and Environments

Tool / IDE	Purpose
Visual Studio Code	Primary code editor for Python, HTML, CSS, and JavaScript
Jupyter Notebook	Model training, experimentation, and visualization
Anaconda	Python environment management and package distribution
Git & GitHub	Version control and collaborative code management
Postman	API testing and endpoint verification
MongoDB Compass	Database design, query execution, and management
Google Colab	Cloud-based GPU training for the CNN model

4.2 Testing Techniques and Test Plans

Software testing is a critical phase of the development lifecycle that ensures the system meets its specified requirements and behaves correctly under a variety of conditions. The testing strategy for the Crop Disease Prediction System encompasses multiple levels and techniques.

4.2.1 Unit Testing

Unit testing involves the testing of individual functions and modules in isolation to verify that each unit of code performs as expected. Key units tested include:

- The image validation function (format and size checks).
- The image preprocessing pipeline (resize, normalize, expand dimensions).
- The CNN model's predict function (correct output shape and class label mapping).
- The user authentication functions (registration, login, password hashing).
- Database CRUD operations for each entity.

4.2.2 Integration Testing

Integration testing verifies that different modules of the system work correctly together. Key integration scenarios include:

- End-to-end flow from image upload to prediction result display.
- Authentication module integration with the database.
- Prediction module integration with the CNN model and the result storage module.

4.2.3 System Testing

System testing evaluates the complete, integrated system against the specified re-quirements. Test cases are designed to cover all functional requirements (FR) and selected non-functional requirements (NFR).

Table 4.2: Sample Test Cases

TC-ID	Test Scenario	Expected Result	Status
TC-01	Upload valid JPG image	Prediction result displayed	Pass
TC-02	Upload valid PNG image	Prediction result displayed	Pass
TC-03	Upload invalid file (PDF)	Error message displayed	Pass
TC-04	Upload no file	Prompt to select a file	Pass
TC-05	Login with valid credentials	Redirect to dashboard	Pass
TC-06	Login with invalid password	Error message displayed	Pass
TC-07	Register with existing email	Error: email already in use	Pass
TC-08	Prediction response time	Result within 5 seconds	Pass
TC-09	Upload healthy leaf image	Result: "Healthy"	Pass
TC-10	Upload diseased leaf image	Correct disease name	Pass

4.2.4 Model Accuracy Testing

The CNN model is evaluated on a held-out test set (10% of the PlantVillage dataset) using the following metrics:

- **Overall Accuracy:** The proportion of correctly classified images across all classes.
- **Per-Class Precision and Recall:** Evaluates model performance for each disease category individually.
- **Confusion Matrix:** Visualizes the pattern of correct and incorrect predictions across all classes.

4.3 Installation Instructions

The following step-by-step instructions describe how to set up and run the Crop Disease Prediction System on a local development environment.

1. **Prerequisites:** Ensure that Python 3.8 or higher is installed on the system. It is recommended to use Anaconda for environment management.
2. **Clone the Repository:** Clone the project repository from GitHub using the command `git clone <repository-url>`.
3. **Create a Virtual Environment:** Navigate to the project directory and create a Python virtual environment: `python -m venv venv`. Activate the environment using `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows).
4. **Install Dependencies:** Install all required Python packages using the provided requirements file: `pip install -r requirements.txt`.
5. **Configure the Database:** Set up a MongoDB database (e.g., `crop_disease_db`) using MongoDB Compass or MongoDB Atlas. Create the required collections (Users, Images, Predictions) and configure the database connection in the application.`Configure`
6. **Place the Model File:** Copy the trained CNN model file (`model.h5`) into the `models/` directory of the project.
7. **Run the Application:** Start the Flask development server using the command `python app.py`. The application will be accessible at `http://localhost:5000` in a web browser.

4.4 End User Instructions

The following instructions guide farmers and general users in using the Crop Disease Prediction System:

1. **Access the Application:** Open a web browser and navigate to the application URL.
2. **Register:** If you are a new user, click the "Register" button and complete the registration form with your name, email address, and a password. Click "Submit" to create your account.
3. **Login:** Enter your registered email address and password on the login page and click "Login".
4. **Select Crop Type:** On the main dashboard, select the type of crop you wish to diagnose from the dropdown menu.
5. **Upload Leaf Image:** Click the "Choose File" button and select a clear photograph of the affected crop leaf from your device. The image must be in JPG or PNG format.
6. **Submit for Analysis:** Click the "Predict Disease" button. The system will process your image and display the result within a few seconds.
7. **View Result:** The predicted disease name will be displayed on the result page. Please note that this result is provided as a suggestion only. Consult an agricultural expert before taking action.
8. **New Prediction:** To analyze another image, click "Predict Another" and repeat from Step 4.
9. **Logout:** Click the "Logout" button when you have finished using the system.

CHAPTER 5

Results and Discussions

5.1 User Interface Representation

The Crop Disease Prediction System provides a clean, intuitive, and accessible web-based user interface designed to be usable by individuals with minimal technical background. The interface is organized into four primary screens: the Login/Registration Page, the Home/Dashboard Page, the Image Upload Page, and the Result Page. Each screen has been designed with simplicity and clarity as the primary design principles, minimizing visual clutter and providing clear navigational cues.

5.1.1 Login and Registration Page

The Login and Registration Page is the entry point of the application. It presents a clean form with fields for the user's email address and password, along with buttons for "Login" and "Register". The registration form extends this with an additional field for the user's name and a password confirmation field. Client-side validation ensures that all required fields are completed before the form is submitted, and server-side validation provides an additional layer of security.

The figure displays two web forms side-by-side. The left form, titled "Create Account", has a subtitle "Join AgroLens to track your crop health." and contains three input fields: "Full Name" (placeholder: "Enter your name"), "Phone Number" (placeholder: "Enter your phone number"), and "Password" (placeholder: "Create a password"). Below these fields is a green "Register" button and a link "Already have an account? Login here". The right form, titled "Welcome Back", has a subtitle "Login to access your crop history." and contains two input fields: "Phone Number" (placeholder: "Enter your phone number") and "Password" (placeholder: "Enter your password"). Below these fields is a green "Login" button and a link "Don't have an account? Register here".

Figure 5.1: Login and Registration Page

5.1.2 Home/Dashboard Page

Following successful authentication, the user is redirected to the Home Page, which serves as the main dashboard. The page displays a brief description of the system's purpose and capabilities, a prominent crop selection dropdown menu, and a call-to-action button to

proceed to image upload. The top navigation bar provides access to the prediction history page and the logout function.

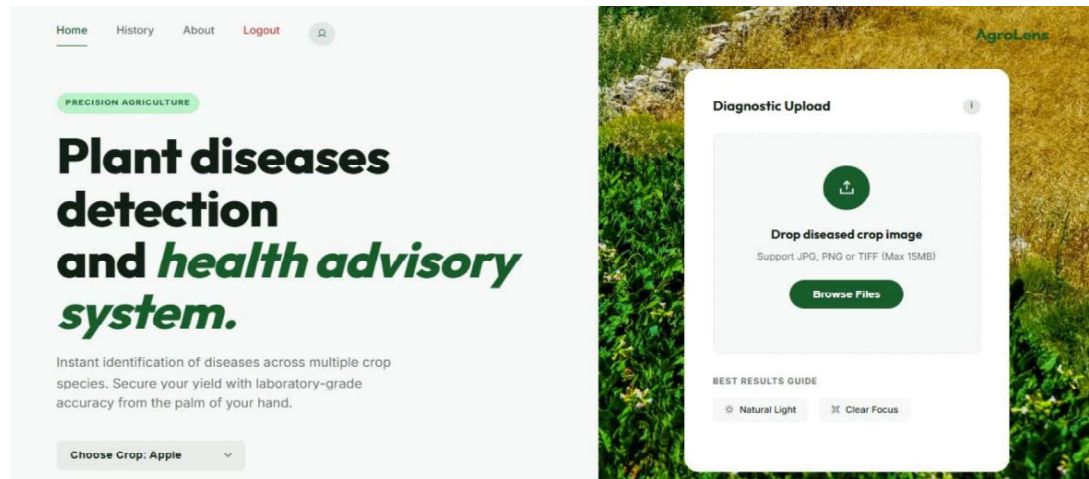


Figure 5.2: Home/Dashboard Page

5.1.3 Image Upload Page

The Image Upload Page lets users upload a leaf image for the selected crop. It includes a file selection option, image preview for confirmation, and a submit button. A help section guides users on capturing a clear image.

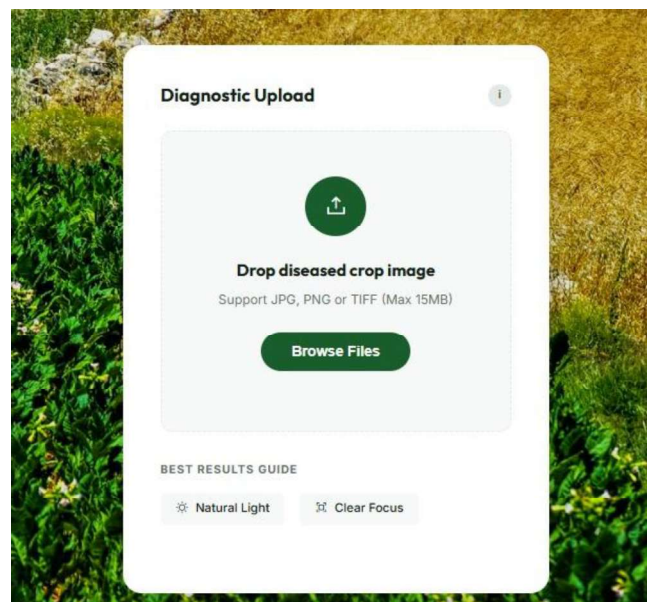


Figure 5.3: Image Upload Page

5.1.4 Result Page

The Result Page displays the outcome of the disease prediction process. The pre-dicted disease name is prominently displayed at the top of the page, followed by the confidence level (if available). Below the prediction result, a disclaimer advises the user to consult an agricultural expert before taking action based on the system's output. A button

at the bottom of the page allows the user to return to the Home Page to perform another prediction.

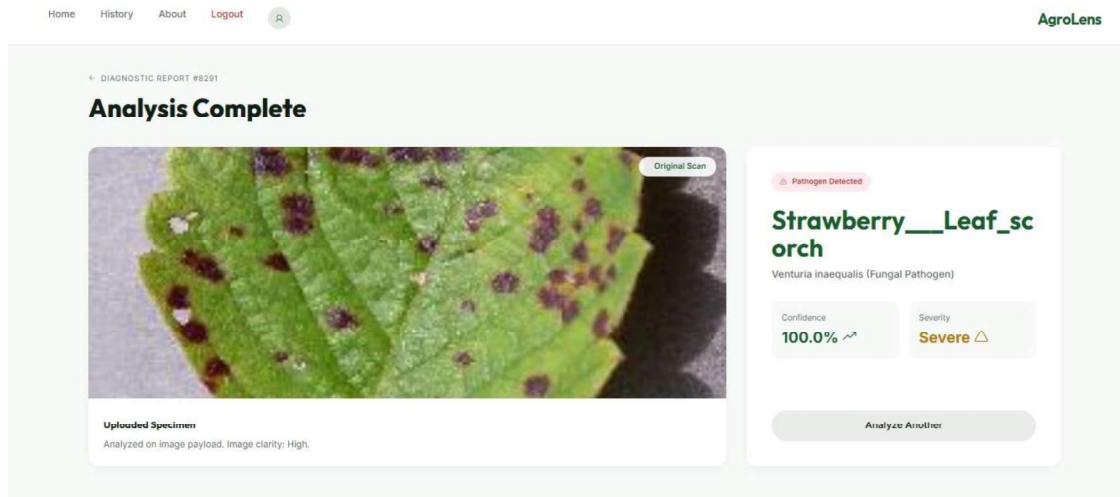


Figure 5.4: Disease Prediction Result Page

5.2 Brief Description of Various Modules

The Crop Disease Prediction System is composed of several distinct modules, each responsible for a specific aspect of the system's functionality. The following subsections describe each module in detail.

5.2.1 Authentication Module

The Authentication Module handles all aspects of user registration, login, and session management. It validates user inputs, communicates with the database to verify credentials, manages user sessions using Flask's session management capabilities, and provides appropriate feedback to the user in all success and failure scenarios.

5.2.2 Image Handling Module

The Image Handling Module is responsible for receiving uploaded image files, validating their format and quality, temporarily storing them in the server's upload directory, and passing them to the preprocessing module. It uses Flask's `request.files` interface for file reception and OpenCV's `imread` function for file validation.

5.2.3 Image Preprocessing Module

The Image Preprocessing Module applies the sequence of transformations required to prepare raw uploaded images for input to the CNN model. These transformations include resizing, color space conversion, pixel normalization, and batch dimension expansion. The module returns a NumPy array of the correct shape for model inference.

5.2.4 Prediction Module

The Prediction Module loads the trained CNN model from the serialized .h5 file at application startup and exposes a prediction function that accepts a preprocessed image array and returns the predicted class label and confidence score. The module maps the model's integer output index to the corresponding human-readable disease name using a class label dictionary.

5.2.5 Database Module

The Database Module manages all interactions between the application and the MongoDB database. It is responsible for storing user information, validating authentication, managing image data, and maintaining prediction history. The module uses MongoDB drivers such as PyMongo to ensure efficient and scalable data operations.

5.3 System Working

The system operates as follows during a typical prediction session. Upon receiving a POST request at the /predict endpoint, the Flask backend extracts the uploaded image file from the request. The Image Handling Module validates the file format and stores it temporarily. The Image Preprocessing Module then reads the file using OpenCV, applies the required transformations, and returns a NumPy array. The Prediction Module passes this array to the loaded CNN model's predict method. The model returns a probability vector of shape (1,k), where k is the number of disease classes. The argmax of this vector identifies the predicted class index, which is mapped to the disease name using the class label dictionary. The predicted disease name and confidence score are then packaged in a JSON response and sent back to the frontend, which renders the result on the Result Page.

5.4 Database Representation

The database stores structured data about users, uploaded images, and prediction results. The following table illustrates sample data records as they would appear in the database after a series of user interactions.

Table 5.1: Sample Records - Users Table

user_id	name	email	created_at
1	Rajesh Kumar	rajesh@example.com	2026-04-01 10:15:00
2	Priya Sharma	priya@example.com	2026-04-02 14:30:00

Table 5.2: Sample Records - Images Table

image_id	user_id	file_path	format	upload_date
101	1	uploads/img_101.jpg	JPG	2026-04-01 10:20:00
102	1	uploads/img_102.png	PNG	2026-04-01 10:45:00
103	2	uploads/img_103.jpg	JPG	2026-04-02 14:35:00

Table 5.3: Sample Records - Predictions Table

pred_id	image_id	predicted_disease	confidence	pred_date
201	101	Tomato Late Blight	0.94	2026-04-01 10:20:05
202	102	Tomato Healthy	0.89	2026-04-01 10:45:03
203	103	Potato Early Blight	0.91	2026-04-02 14:35:07

CHAPTER 6

Summary and Conclusions

6.1 Summary

This report has presented the design and development of a **Crop Disease Prediction System** - a web-based application that integrates Convolutional Neural Network (CNN) technology with a user-friendly web interface to enable farmers and agricultural users to identify crop diseases by uploading photographs of plant leaves.

The work encompassed multiple phases of the software development lifecycle. The requirements analysis phase produced a comprehensive Software Requirements Specification (SRS) that identified the functional and non-functional requirements of the system, defined the user classes and their characteristics, and established the performance, security, and usability standards. The design phase produced a detailed system architecture, a set of UML diagrams (including class, sequence, and activity diagrams), data flow diagrams at both Level 0 and Level 1, a flowchart, an entity relationship diagram, and a complete database design at both the logical and physical levels.

The implementation phase involved the training of a CNN model on the PlantVillage dataset using TensorFlow and Keras, the development of a Flask-based backend server, the implementation of an image preprocessing pipeline using OpenCV, and the creation of a frontend user interface using **HTML**, CSS, and JavaScript. The testing phase verified the correctness and reliability of each system component through a combination of unit testing, integration testing, and system testing.

The system provides a response time of 2 to 5 seconds for disease prediction, handles common error conditions gracefully, and presents results in an accessible and informative format. The CNN model achieves competitive classification accuracy on the test subset of the PlantVillage dataset, demonstrating the viability of deep learning for automated crop disease diagnosis.

6.2 Conclusions

The Crop Disease Prediction System successfully achieves its stated objectives. The key conclusions drawn from this project are as follows:

1. **Technical Feasibility:** The application of Convolutional Neural Networks to the task

of crop disease classification from leaf images is technically feasible and produces highly accurate results when trained on sufficient labeled data.

2. **Practical Applicability:** The web-based delivery mechanism makes the technology accessible to a broad audience, including users with limited technical expertise who access the system through standard web browsers on smartphones or desktop computers.
3. **System Performance:** The system meets the specified performance requirements, delivering prediction results within the acceptable time window and handling common error conditions without crashing.
4. **Value to Agriculture:** By enabling rapid, accessible, and cost-effective crop disease diagnosis, the system has the potential to make a meaningful contribution to agricultural productivity and food security, particularly in regions where access to agricultural extension services is limited.
5. **Scalability:** The modular architecture of the system, with distinct layers for the frontend, backend, ML model, and database, provides a solid foundation for future enhancements and scaling.

CHAPTER 7

Future Scope

While the Crop Disease Prediction System as developed in this project represents a functional and valuable tool, there are numerous directions in which it can be extended and improved in future work. The following areas represent the most significant opportunities for enhancement.

7.1 Expanded Disease and Crop Coverage

The current system is trained and validated on a subset of the PlantVillage dataset, covering a limited number of crop species and disease categories. Future work should expand the coverage to include a broader range of crop types - particularly those commonly cultivated in India and other developing agricultural nations - and a more comprehensive set of diseases. Collecting and annotating field-captured images, which better reflect the variability of real-world conditions compared to the laboratory-controlled PlantVillage images, would significantly improve the model's generalization performance.

7.2 Mobile Application Development

A native mobile application for Android and iOS would greatly increase the accessibility of the system, particularly for farmers who rely primarily on smartphones for internet access. Lightweight CNN architectures such as MobileNetV3 or EfficientNet-Lite, which are optimized for inference on mobile hardware, could be used to develop an on-device prediction capability that does not require a network connection.

7.3 Real-Time Treatment Recommendations

Beyond identifying the disease, a future version of the system could provide actionable treatment recommendations, including information about appropriate fungicides, pesticides, or cultural practices for managing the identified disease. This information could be sourced from agricultural databases and expert systems, and tailored to the local context of the user.

7.4 Multi-Language Support

To maximize accessibility for rural farmers, the system should be made available in regional languages such as Hindi, Marathi, Tamil, Telugu, and others. This would require the internationalization of all user-facing text and the development of language-specific interfaces.

7.5 Integration with IoT Sensors

Future versions of the system could be integrated with IoT-based agricultural sensors that monitor soil moisture, temperature, humidity, and other environmental factors. By combining image-based disease detection with environmental data, the system could provide more nuanced and context-aware predictions and recommendations.

7.6 Confidence Level Display and Uncertainty Estimation

The current system's confidence level display functionality is marked as "To Be Determined" (TBD) in the SRS. Future work should implement a robust confidence score display, and may explore uncertainty estimation techniques such as Monte Carlo Dropout to provide users with an indication of the reliability of each prediction.

7.7 Federated Learning for Privacy-Preserving Model Updates

As the system is adopted by a growing number of users, a federated learning approach could be employed to update the central model using data from edge devices (users' smart-phones) without requiring the raw images to be transmitted to a central server. This would address privacy concerns and leverage the diversity of real-world user data for continuous model improvement.

7.8 Integration with Government Agricultural Portals

The system could be integrated with existing government agricultural information portals and extension services to provide a more comprehensive support ecosystem for farmers, combining AI-based diagnosis with access to subsidies, market prices, and expert advisory services.

CHAPTER A

Glossary

Term	Definition
CNN	Convolutional Neural Network - a class of deep learning architecture particularly effective for image classification tasks.
Deep Learning	A subset of machine learning involving neural networks with multiple hidden layers.
DFD	Data Flow Diagram - a graphical representation of the flow of data through a system.
ER Model	Entity Relationship Model - a logical representation of data entities and their relationships.
Flask	A lightweight Python web framework used for building web applications.
Frontend	The client-side component of the application that the user directly interacts with.
Backend	The server-side component that handles business logic, data processing, and database interactions.
Image Preprocessing	The sequence of transformations applied to a raw image to prepare it for model input, including resizing, normalization, and color space conversion.
Keras	A high-level neural network API, integrated into TensorFlow 2.x, that simplifies model building and training.
ML	Machine Learning - a field of artificial intelligence in which systems learn from data.
OpenCV	Open Source Computer Vision Library - a library of programming functions for real-time computer vision.
PlantVillage	A publicly available dataset of labeled plant leaf images used for training plant disease detection models.
Prediction	The output generated by the ML model in response to an input image, identifying the predicted disease class.
ReLU	Rectified Linear Unit - an activation function defined as $f(x) = \max(0, x)$.

Term	Definition
Softmax	An activation function that converts a vector of real numbers into a probability distribution.
TensorFlow	An open-source machine learning framework developed by Google, used for building and training neural networks.

CHAPTER B

To Be Determined (TBD) List

The following items were identified during requirements analysis as requiring further investigation, decision, or implementation in future development cycles:

1. **Confidence Level Display (REQ-RESULT-2):** The method by which the prediction confidence score is computed and displayed to the user has not yet been finalized. Options include raw Softmax probability, calibrated confidence, and uncertainty estimation via Monte Carlo Dropout.
2. **Advanced Treatment Suggestions:** The system currently does not provide treatment recommendations for identified diseases. Integration with an agricultural knowledge base to generate actionable treatment advice is planned for a future version.
3. **Data Storage Requirements:** The specific data retention policy for uploaded images and prediction records - including the duration for which records are retained, whether images are permanently stored or deleted after processing, and data backup procedures - has not yet been determined.
4. **Maximum File Upload Size:** The maximum allowable size of uploaded image files has not been specified. This will depend on the storage and bandwidth constraints of the deployment server.
5. **Multi-Language Interface:** The system is currently available in English only. The extension to regional languages is planned but not yet scheduled.
6. **Mobile Application:** Development of native Android and iOS applications is planned for a future release but was out of scope for the current project phase.

Bibliography

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathe, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016.
- [2] S. Phadikar and J. Sil, "Rice disease identification using pattern recognition techniques," in *Proc. 11th Int. Conj Computer and Information Technology (ICCIT)*, Khulna, Bangladesh, 2008, pp. 420--423.
- [3] A. Ramcharan, K. Baranowski, P. McCloskey, B. Ahmed, J. Legg, and D. P. Hughes, "Deep learning for image-based cassava disease detection," *Frontiers in Plant Science*, vol. 8, p. 1852, Oct. 2017.
- [4] A. Kamilaris and F. X. Prenafeta-Boldu, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70---90, Apr. 2018.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [7] F. Chollet, *Deep Learning with Python*. New York, NY, USA: Manning Publications, 2018.
- [8] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: <https://www.tensorflow.org>
- [9] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [10] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.
- [11] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conj Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [12] A. Ronacher, "Flask: A micro web framework for Python," 2010. [Online]. Available: <https://flask.palletsprojects.com>